

BASELINE · SECURITY REVIEW

Baseline AMM Review.

A security review of AMM Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Baseline	AMM Review	Guardian	August 27 – September 13, 2025	September 25, 2025

§ 00

Contents.

72 pages — overview, confidence ranking, scope, 54 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 54 findings	P. 11
§ 05	Disclaimer	P. 71
§ 06	About Guardian	P. 72

§ 01

Overview.

Guardian reviewed AMM Review across a single review round. The final report includes 3 Critical, 6 High, 13 Medium, 11 Low, 21 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	0xBaseline/mercury
CLIENT	METHODOLOGY
Baseline	Smart contract · Manual · Main Review
PROJECT	MAIN REVIEW COMMIT
AMM Review	e3dab29adae8
PREPARED BY	
Guardian	
AUDIT WINDOW	
August 27 – September 13, 2025	
FINAL REPORT	
September 25, 2025	

1.1 Confidence Ranking

2 / 5

Low Confidence

Guardian assigns a Confidence Ranking of 2 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	3	0	3	0	0
● High	6	0	6	0	0
● Medium	13	0	13	0	0
● Low	11	0	11	0	0
● Informational	21	0	21	0	0
All severities	54	0	54	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence ASSIGNED	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed AMM Review in `0xBaseline/mercury`. The scope includes 2,525 source lines across 15 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/ActionRouter.sol	125	184
src/BProxy.sol	94	144
src/BToken.sol	15	21
src/libraries/AllocatorLib.sol	154	267
src/libraries/NativeLib.sol	37	51
src/libraries/ProxyLib.sol	26	36
src/libraries/StateLib.sol	117	182
src/libraries/SweepLib.sol	41	68
src/components/BAllocator.sol	83	117
src/components/BCredit.sol	386	622
src/components/BFactory.sol	126	186
src/components/BHook.sol	163	214
src/components/BLens.sol	359	605
src/components/BSwap.sol	772	1,371
src/utils/ConfigScript.sol	27	33

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Sells Can Be Used To Drain The Pool	Gaming	CRITICAL	ACKNOWLEDGED	p. 11
C-02	BLV Inflation Attack	Gaming	CRITICAL	ACKNOWLEDGED	p. 13
C-03	Wrong Calculations During Vault Enter/Exit	Logical Error	CRITICAL	ACKNOWLEDGED	p. 15
H-01	Invalid costBasis Tracking	Logical Error	HIGH	ACKNOWLEDGED	p. 17
H-02	Average Price Assignment Allows Arbitrage	Gaming	HIGH	ACKNOWLEDGED	p. 18
H-03	Sweep Does Not Work	Logical Error	HIGH	ACKNOWLEDGED	p. 20
H-04	Missing Slippage Validation	Validation	HIGH	ACKNOWLEDGED	p. 21
H-05	Yield Loss Via Claim Function Not Harvesting	Rewards	HIGH	ACKNOWLEDGED	p. 22
H-06	Missing Check In updateBlvPrice	Validation	HIGH	ACKNOWLEDGED	p. 23
M-01	Incorrect Hook Mask In Deployment	Logical Error	MEDIUM	ACKNOWLEDGED	p. 24
M-02	Outstanding Reserves May Remain Stuck	DoS	MEDIUM	ACKNOWLEDGED	p. 25
M-03	Repay DoS Griefing Attack	DoS	MEDIUM	ACKNOWLEDGED	p. 26
M-04	Conflicting Deposit Or Withdrawal Fee Vaults	Warning	MEDIUM	ACKNOWLEDGED	p. 27
M-05	Re-org BToken Deployment Risk	—	MEDIUM	ACKNOWLEDGED	p. 28

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-06	Mismatching Fees For Leverage And Borrow	Logical Error	MEDIUM	ACKNOWLEDGED	p. 29
M-07	Missing Routes	Logical Error	MEDIUM	ACKNOWLEDGED	p. 30
M-08	All BTokens Cannot Be Sold	Logical Error	MEDIUM	ACKNOWLEDGED	p. 31
M-09	XSS Attack With Unsanitized Token Details	Validation	MEDIUM	ACKNOWLEDGED	p. 33
M-10	Inconsistent Fee Charged	Logical Error	MEDIUM	ACKNOWLEDGED	p. 34
M-11	sqrtPriceLimit Ignored	Warning	MEDIUM	ACKNOWLEDGED	p. 35
M-12	Missing Sweep In Borrow Flow	DoS	MEDIUM	ACKNOWLEDGED	p. 36
M-13	Yield Distributed To Stakers If There Aren't Any	Rewards	MEDIUM	ACKNOWLEDGED	p. 37
L-01	poolManager Set Twice	Gas Optimization	LOW	ACKNOWLEDGED	p. 38
L-02	Fee Claiming May Be Blocked By Zero Amounts	DoS	LOW	ACKNOWLEDGED	p. 39
L-03	Protocol Fee Claim Can Be DoS	DoS	LOW	ACKNOWLEDGED	p. 40
L-04	Claimers Miss Yield Harvest	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 41
L-05	takeReserves Does Not Refund Accidental Eth	Validation	LOW	ACKNOWLEDGED	p. 42
L-06	Inessential ensureValidCurve Convexity Validation	Validation	LOW	ACKNOWLEDGED	p. 43
L-07	Misleading Staking Percent	Logical Error	LOW	ACKNOWLEDGED	p. 44
L-08	Missing Deployment Validation Functions	Warning	LOW	ACKNOWLEDGED	p. 45
L-09	Fee Claiming May Be Blocked By Protocol	DoS	LOW	ACKNOWLEDGED	p. 46
L-10	Frequent Claims Slow Staking Distribution	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 47

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-11	beforeModifyLiquidity Revert Missing	Best Practices	LOW	ACKNOWLEDGED	p. 48
I-01	Misleading Comment	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 49
I-02	Redundant Executor Assignment	—	INFORMATIONAL	ACKNOWLEDGED	p. 50
I-03	Unused getCollateralForBorrow Function	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 51
I-04	Admin Can Be Added As An Executor	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 52
I-05	Version Limits Upgrades	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 53
I-06	Lacking executeActions Return Value	—	INFORMATIONAL	ACKNOWLEDGED	p. 54
I-07	Incorrect Comment	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 55
I-08	Component Contracts Must Report Static Labels	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 56
I-09	Admin Can Be Removed As An Executor	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 57
I-10	Maximum ContractName Label Length	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 58
I-11	Lacking Zero Address Checks	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 59
I-12	Unnecessary Return Value	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 60
I-13	Misleading shareBalance Comment	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 61
I-14	Risk Of Insolvency And Bank Run	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 62
I-15	Rounding Risk	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 63
I-16	Accumulator Update Skips	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 64
I-17	Redundant Check	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 66
I-18	Vault Loss Will Break Calcs	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 67

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-19	Lending Is Capital Inefficient	Rewards	INFORMATIONAL	ACKNOWLEDGED	p. 68
I-20	Variable Missing In NatSpec	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 69
I-21	Unused Code	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 70

CRITICAL **C-01**

ACKNOWLEDGED

Sells Can Be Used To Drain The Pool

CATEGORY	Gaming
LOCATION	BSwap.sol
REVIEW	Main Review

— DESCRIPTION

Because of the nature of the Baseline AMM, where the curve shifts upon every trade and uses a different convexity for the bid side versus the ask side, there is an arbitrage opportunity where a trader sells their BTokens and immediately buys them back at a lower price. This arbitrage can be abused to effectively drain the pool.

The issue stems from the convexity (slippage) difference. If a user buys BTokens from the pool, if they attempt to sell them back to the pool they receive less reserves than they originally bought them with if the bid curve has greater convexity (more slippage). However on the flip side, when the bid curve has greater convexity than the ask curve of the previous snapshot there is an opportunity for arbitrage in the opposite direction by shorting the BToken.

Any BToken holder can sell their BTokens on the higher slippage bid curve and then buy them back using the lower slippage ask curve. Since the ask curve has lower slippage they can buy their original BToken holdings back at a lower average price and arbitrage risk free profits.

The swap fee can counteract the profitability of these trades, however arbitrages still exist at 25-30 basis points of a swap fee and likely even higher.

— RECOMMENDATION

A lot of issues around arbitrage and overall execution price gaming will stem from the snapshot system and the ability of the AMM curve to change after every swap. If the dynamic AMM curves must be kept, then these arbitrages and gaming vectors may be alleviated by allowing curves to adjust over a period of time rather than instantaneously.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/a1fa3550763a03e105bc0039b5141b36#file-shortarb-t-sol-L167>

C-01 Sells Can Be Used To Drain The Pool

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

C-02 BLV Inflation Attack

CONTINUED

```
1000000000000000000000000000000000000 Pool Reserves
10100000000000000000000000000000000000000000000000000000000000000000000000 Pool Active Price
100000000000000000000000000000000000000000000000000000000000000000000000 BLV Price
100000000000000000000000000000000000000000000000000000000000000000000000 Book Price
100000000000000000000000000000000000000000000000000000000000000000000000 blv Reserves
0 buffer Reserves
-----
```

— RECOMMENDATION

Consider capping the amount that BLV can increase per block or period of time to a modest percent.

— RESOLUTION

Status: Acknowledged.

CRITICAL **C-03**

ACKNOWLEDGED

Wrong Calculations During Vault Enter/Exit

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/mercury-baselineamm-team2/blob/e3dab29adae84e32a72c8b51634bd9e7cb0173e4/src/components/BController.sol#L59-L79
REVIEW	Main Review

DESCRIPTION

When a pool with vault is deployed and users interact with it the amount of tokens in the vault are:

$$\text{vaultBalance} = \text{totalReserves} - \text{totalDebt} + \text{unclaimedFees} + \text{unrecordedVaultYield}$$

However when the admin switches to a new vault or exits the vault this no longer is the case:

1. Exit

The exit flow is the following:

- All reserves are redeemed from the vault
- The `totalReserves` variable is updated to: $\text{totalReserves} = \text{redeemed} + \text{totalDebt} - \text{unclaimedFees}$

Here lies the first issue. As we can see in the first formula `totalReserves` is increased by the `unrecordedVaultYield`. Therefore this yield is stolen from the staker and the book price suddenly increases which allows MEV attacks.

1. Enter

The new vault is entered by depositing the `totalReserves`.

This is the second issue. As we can see in the first formula the vault balance is not supposed to be `totalReserves`.

This leads to many issues:

- Potential DoS as it tries to deposit `totalDebt` into the vault (which is not inside the contract)
- `unclaimedFees` are not deposited into the vault and they are left inside the baseline contract. This has multiple negative effects, for example:
- It messes with future yield as in the `getHarvestableYield` calculation the `holdings` stay the same but the `assetsWithYield` decreased. Therefore future yield is not tracked / stolen from stakers for a while.

C-03 Wrong Calculations During Vault Enter/Exit

CONTINUED

• As the vault is connected to the pool the `giveReserves` function will always try to withdraw the given reserves from the vault however `unclaimedFees` are now part of calculations but aren't deposited in the vault. This can therefore lead to DoS.

However the biggest issue lies in the recalculation of `totalReserves` based on the `redeemed` amount (during exit). If any reserve tokens were not deposited in the vault during that time the resulting `totalReserves` is wrong (smaller than before). This happens if tokens lay inside Uniswap and aren't swept yet or as already mentioned if a vault switch already happened in the past as the `unclaimedFees` lay inside the Baseline contracts in that case.

The impact of the decreased `totalReserves` value is drastical as the book price will be influenced and critical invariants like for example: `totalReserves >= circSupply * BLV` can break.

And of course malicious actors could trigger it on purpose by performing a swap on Uniswap right before a vault switch.

— RECOMMENDATION

1. Sweep tokens on Uniswap and claim yield before entering or exiting a vault.
2. Deposit `totalReserves - totalDebt + unclaimedFees` instead of `totalReserves` into the new vault.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/ae96bdb3c40eb83bb661cdd7529060e487e1000a/test/fuzzing/FoundryPlayground.sol#L713-L746>

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/12595345c4aa8b311b76b501999a26e99f9990a9>

HIGH **H-01**

ACKNOWLEDGED

Invalid costBasis Tracking

CATEGORY	Logical Error
LOCATION	BSwap.sol
REVIEW	Main Review

DESCRIPTION

The `updatePosition` function incorrectly tracks the `costBasis` of the pool by assuming that the circulating supply has not been updated for the current swap at the time of invocation.

In the case where a trader is buying BTokens from the pool and the `bTokenDelta` is less than zero, the `newCircSupply` is calculated as the `circSupply + tokenAmount` in an attempt to create the new circulating supply of the system after the swap. However the swap has already been recorded in the pool amount and thus is already reflected in the `circSupply` value.

Instead the `costBasis` calculation needs to compute the circulating supply from before the buy occurred to get the reserves that were already used in the `costBasis`.

RECOMMENDATION

Replace the circulating supply logic and `costBasis` calculation in the buy case with the following:

```
if (bTokenDelta < 0) {  
    uint256 oldCircSupply = circSupply - tokenAmount;  
    mkr.costBasis = (oldCircSupply * mkr.costBasis + tokenAmount * executionPrice) / circSupply;  
}
```

RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

Average Price Assignment Allows Arbitrage

CATEGORY Gaming
LOCATION BSwap.sol
REVIEW Main Review

DESCRIPTION

Because the Baseline AMM curve shifts with each swap, there is a superior buying strategy which allows users to obtain BTokens at a lower average price by splitting up larger buys into multiple smaller ones, to take advantage of the way that the curve changes its convexity and active price.

For example, consider the following test cases and outputs:

```
function test_buyTokensFull() public {
  uint256 bTokensOut = 10_000 * WAD;
  vm.prank(alice);
  uint256 actualReservesIn = bSwap.buyTokens(bToken, bTokensOut, WAD * 100_000);
  console.log("Total reserves in:", actualReservesIn); // 10,531.578947368421060000
}

function test_buyTokensSplit() public {
  uint256 bTokensOut = 5_000 * WAD;
  vm.startPrank(alice);
  uint256 actualReservesIn = bSwap.buyTokens(bToken, bTokensOut, WAD * 100_000);
  actualReservesIn += bSwap.buyTokens(bToken, bTokensOut, WAD * 100_000);
  vm.stopPrank();
  console.log("Total reserves in:", actualReservesIn); // 10,395.310728744939278860
}
```

In the first test case, the total cost to obtain 10,000 BTokens is 10,531.57 reserves, however just splitting this buy up into 2 smaller buy swaps allows the user to obtain 10,000 BTokens for 10,395.31 reserves which is ~1.3% less reserves required. This difference can increase with more chunked buys and larger buys relative to the pool's liquidity.

This occurs mainly because the price of the AMM is assigned as the average price of the swap that was previously executed, which affects the execution price of the following swap. This behavior means that the pool starts to follow a different path depending on the size, direction, and order of swaps that are executed in the AMM. Unlike a typical CPMM where X number of buy volume and Y number of sell volume will always result in the same price

H-02 Average Price Assignment Allows Arbitrage**CONTINUED**

regardless of ordering (assuming full range liquidity), the Baseline AMM will have a different outcome for every permutation of action which is extremely gameable. Malicious actors can simply create orderings with buy volume that yields a lower average entry price than a subsequent ordering of the same magnitude sell volume and yield immediate gains against the AMM.

— RECOMMENDATION

Consider reconstructing the logic of the AMM such that the price follows one path as a function of the reserves being made available rather than the average execution price of previous swaps.

— RESOLUTION

Status: Acknowledged.

HIGH **H-03**

ACKNOWLEDGED

Sweep Does Not Work

CATEGORY Logical Error
LOCATION BHook.sol
REVIEW Main Review

DESCRIPTION

There is an external `sweep` function on the `BHook` contract which allows anyone to sweep the outstanding funds in the Uni V4 hook if there are any.

However, this function cannot be used since the pool manager `unlockCallback` function attempts to call the

`executeSweep` function on itself.

The `executeSweep` function requires that the caller is the `poolManager` and therefore this function being called from the `unlockCallback` will fail the `onlyPoolManager` validation.

RECOMMENDATION

Consider making the `executeSweep` external function on the `BHook` contract callable by `address(this)` only, or using `delegatecall` instead of `call` in the `unlockCallback`.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/b8f90f72f1bb81486ff6d4aa6debfd9a8194e343>

HIGH **H-04**

ACKNOWLEDGED

Missing Slippage Validation

CATEGORY	Validation
LOCATION	BHook.sol
REVIEW	Main Review

DESCRIPTION

In the `BHook._swap` function there is no slippage validation as the `limit` is assigned to the extremes of 0 or `type(uint256).max`.

As a result users can be frontrun to extract significant value from their trades through the `BHook`. Furthermore, even if there is no public mempool on a network that mercury is being used on, a user may simply receive a worse execution than they were expecting purely based on the amount of time it takes their order to be included in a block and the market activity that occurs before that.

RECOMMENDATION

Consider adding support for a slippage parameter in the form of a `maxIn/minOut` that can be passed along by the user through the `hookData` in the `swap` function.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/c373edb12b140af393e9d2c7a87bd0df01321466>

HIGH **H-05**

ACKNOWLEDGED

Yield Loss Via Claim Function Not Harvesting

CATEGORY Rewards

LOCATION <https://github.com/GuardianOrg/mercury-baselineamm-team1/blob/e3dab29adae84e32a72c8b51634bd9e7cb0173e4/src/components/BStaking.sol#L132-L133>

REVIEW Main Review

DESCRIPTION

In the Baseline staking module, calling `claim` only updates the timestamp of `lastUpdated` but does not harvest or distribute accumulated yield. As a result, the `pendingYield` or accumulator is not updated, effectively discarding yield accrued up to that point. Since `claim` is publicly callable, a malicious actor can repeatedly call it to force Baseline to lose yield on behalf of all users, leading to systemic losses.

RECOMMENDATION

Consider modifying the `claim` function to distribute fees and harvest before executing `_sync`.

RESOLUTION

Status: Acknowledged.

HIGH **H-06**

ACKNOWLEDGED

Missing Check In updateBlvPrice

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/mercury-baselineamm-team2/blob/e3dab29adae84e32a72c8b51634bd9e7cb0173e4/src/components/BSwap.sol#L661-L672>

REVIEW Main Review

— DESCRIPTION

The `updateBlvPrice` function makes sure that the given BLV price is not above the active price. However it does not check if the given BLV exceeds the current book price. This would break critical invariants and can happen even if the admin handles this update carefully as the book price fluctuates.

— RECOMMENDATION

Make sure that the give BLV price is below current book price.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/dddbfb113a39406117f9d8ecdb9b25153b1d0c36/test/fuzzing/FoundryPlayground.sol#L824-L908>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Incorrect Hook Mask In Deployment

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/mercury-baselineamm-team1/blob/e3dab29adae84e32a72c8b51634bd9e7cb0173e4/script/Deploy.s.sol#L76
REVIEW	Main Review

DESCRIPTION

Deployment script uses `Hooks.ALL_HOOK_MASK` when deploying `BHook`, advertising all hooks as implemented to `PoolManager`. Since `BHook` does not implement `afterInitialize`, `createPool` reverts and no pool can be deployed. Tests don't catch this because they configure the hook mask correctly.

RECOMMENDATION

Use only the specific hook flags actually implemented by `BHook` instead of `Hooks.ALL_HOOK_MASK`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Outstanding Reserves May Remain Stuck

CATEGORY	DoS
LOCATION	BHook (Global)
REVIEW	Main Review

— DESCRIPTION

If a swap is attempted through the Uniswap router, Baseline records outstanding reserves, expecting that later opposing flow will allow a sweep. However, there is no guarantee this will occur. `sellTokens` can continue happening through the Uniswap router while `buyTokens` are routed through the Baseline router, keeping the outstanding reserves intact.

Even if most orders happen via Uniswap, sweep is not performed at the exact step—it waits until the full opposing amount is present. For example:

- Eve buys 500 BTs over Baseline for 500 USDC
- Eve sells 500 BTs over Uniswap for 500 USDC
- Bob buys 100 BTs over Uniswap for 100 USDC

At this point, 100 USDC remain stuck, as the system waits for the remaining 400 USDC on Uniswap before a sweep can occur. Ideally, the outstanding should have been cleared right after Eve's sell of 500 BTs. If other actions happen in between, the sweep will not trigger, since it requires the full amount.

This would also lead to underutilization of assets, since assets in pool manager cannot be rehypothecated diluting yield.

— RECOMMENDATION

Consider allowing partial sweeps so that reserves can be cleared incrementally instead of requiring the full amount to match.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Repay DoS Griefing Attack

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/mercury-baselineamm-team2/blob/e3dab29adae84e32a72c8b51634bd9e7cb0173e4/src/components/BCredit.sol#L219
REVIEW	Main Review

DESCRIPTION

The `repay` flow reverts if a user tries to repay more than the full debt of the given position.

This enables a griefing vector:

- Bob tries to repay the full debt of his position
- Eve front runs the call and repays 1 wei of the debt of Bob's position
- Bob's transaction reverts as he tries to repay 1 wei too much

RECOMMENDATION

Consider repaying the maximum possible in case a user tries to repay more debt than the position has.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

Conflicting Deposit Or Withdrawal Fee Vaults

CATEGORY Warning
LOCATION Global
REVIEW Main Review

— DESCRIPTION

In many ERC4626 vaults there may exist deposit or withdrawal fees which are levied to disincentivize sandwich attacking yield gains which cause stepwise changes in share price.

These vaults are incompatible with the Baseline system as they open up a vector for a user to continuously borrow and repay to have the vault levy a significant amount of fees on the protocol, causing significant grief to the stakers and even causing the pool liquidity amounts to be unbacked.

— RECOMMENDATION

Be sure to not use the mercury system with any vaults that have a deposit or withdrawal fee.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Re-org BToken Deployment Risk

CATEGORY —
LOCATION BFactory.sol
REVIEW Main Review

DESCRIPTION

In the `BFactory` contract the `createBToken` function uses the `CREATE` opcode to deploy a new BToken which can then be used to deploy a pool. Since the normal `CREATE` opcode is used for deployment, the resulting BToken address is not unique to the deployer and is dependent on the ordering of deployments made by the factory due to the monotonically increasing account nonce.

As a result, in the event of a Re-org it is possible for a malicious actor to siphon funds from a deployer if they had sent a combination of certain transactions in a short period which were perturbed by the Re-org.

RECOMMENDATION

Use `CREATE2` with the `msg.sender` as a salt in the `createBToken` function.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/5ac229c75f99d46a47bade982a66821b153d241d>

MEDIUM **M-06**

ACKNOWLEDGED

Mismatching Fees For Leverage And Borrow

CATEGORY Logical Error
LOCATION BCredit.sol
REVIEW Main Review

DESCRIPTION

The fee calculation used in the borrow function is different than the fee calculation used in the leverage function, which allows users to borrow at a lower fee using the leverage function.

In the borrow function, the fee calculation is $\text{amount} / (1 - \text{fee}) - \text{amount}$.

In the leverage function, the fee calculation is $\text{amount} * \text{fee}$.

For example:

- amount = 100
- fee = 10%
- Borrow fee = $100 / 0.9 - 100 = 11.111$
- Leverage fee = $100 * 0.1 = 10$

RECOMMENDATION

Consider standardizing on one fee calculation or the other for both leverage and borrow functions.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

Missing Routes

CATEGORY Logical Error
LOCATION Global
REVIEW Main Review

DESCRIPTION

The following function routes have not been configured in each Component:

`BCredit.previewDepositAndBorrow``BLens.poolIdToBToken``BLens.hasHook``BLens.poolKey``BStaking.withdrawAndClaim`

RECOMMENDATION

Add the necessary routes for these functions in their respective components.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/7288b7a01ff700d963eef489449715b97132f7c1>

MEDIUM **M-08**

ACKNOWLEDGED

All BTokens Cannot Be Sold

CATEGORY Logical Error
LOCATION BSwap.sol
REVIEW Main Review

DESCRIPTION

In the `_generateValidPrice` function the `expectedBookPrice` is computed by dividing the new pool reserves by the new BToken circulating supply. However if a user is selling the last circulating BToken to the pool this results in a divide by zero revert, therefore preventing the user from selling the final amount of BTokens.

Furthermore, in the `_updateBlvPrice` function when the final circulating BTokens are being sold, the calculation of the new BLV price panic reverts with a divide by zero revert.

Additionally, In the `ensureValidCurve` function, if the resulting supply in the pool is equal to the total supply, the calculation of the book price will cause a revert.

Finally, the `getBookPremiumRatio` function will revert when the `_bookPrice - _baselineValue` result is 0. This can occur when the not absolutely all of the circulating supply has been sold, but the entire circulating supply less a handful of wei is being sold. In these circumstances where very nearly the entire circulating supply has been sold, the BLV price converges to be the same as the book price and causes a revert when computing the B value. Not only does this prevent sells that leave a handful of wei in the circulating supply by way of reverting in the `ensureValidCurve` function. But it also prevents the remaining wei from being sold because of the use of the `getBookPremiumRatio` function in computing the AMM calculations for that sell.

RECOMMENDATION

Consider early returning with the original price in the `_generateValidPrice` function if there will be no more circulating supply as a result of the BToken sell and because there is no valid book price in this case.

Consider early returning in the `_updateBlvPrice` function when the `circulatingSupply` is 0.

Consider only performing the `bookPrice` validations in the `ensureValidCurve` if the supply in the pool is not the same as the total supply.

M-08 All BTokens Cannot Be Sold

CONTINUED

Consider returning a B value of 1 from the `getBookPremiumRatio` function when the book price is the same as the baseline value price.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

XSS Attack With Unsanitized Token Details

CATEGORY Validation
LOCATION BFactory.sol
REVIEW Main Review

DESCRIPTION

In the `createBToken` function there is no validation on the `_name` and `_symbol` strings which are used for the `BToken`. If this `BToken` is then displayed in the Baseline UI or other consumers of the Baseline pools UI's, then these name and symbol fields could be leveraged for an off-chain XSS attack against users of those applications, or the applications themselves.

This was the case in the EtherDelta exploit, for more details on the EtherDelta exploit refer to this [article] (<https://medium.com/@zhongqiangc/code-injection-attacks-using-blockchains-part-1-db4e121890bb>).

RECOMMENDATION

Consider validating that the length of the name and symbol is below a reasonable limit to prevent malicious XSS injection.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/5ac229c75f99d46a47bade982a66821b153d241d>

MEDIUM **M-10**

ACKNOWLEDGED

Inconsistent Fee Charged

CATEGORY Logical Error
LOCATION BSwap.sol
REVIEW Main Review

DESCRIPTION

In the `computeTokenBuy` and `computeReserveBuy` functions the fee charged is different because of a different calculation method used. As a result, users will receive a lower net fee using the `exactIn` reserve buy functionality compared to the `exactOut` BToken buy functionality.

For example, consider the following scenario:

Fee = 10% = 1e17

Assume price is s.t. 1 BToken == 1 Reserve and there is nearly infinite liquidity relative to trade size so ignore slippage

Buying BTokens With `ExactIn` Reserves

$\text{normalizedReservesIn} = 100$

$\text{normalizedFee} = 100 * 1e17 / 1e18 = 10 = 10\%$

Buying BTokens With `ExactOut` BTokens

$\text{reservesInLessFee} = 100$

$\text{normalizedReservesIn} = 100 * 1e18 / 9e17 = 111.11111111 = 11.11\%$

In this case the fee was 11.111... reserves which does not match the fee rate charged in the other buy function.

RECOMMENDATION

Consider standardizing on one or the other approach for computing the fee so there is no difference.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

sqrtPriceLimit Ignored

CATEGORY	Warning
LOCATION	BHook.sol
REVIEW	Main Review

— DESCRIPTION

In the `beforeSwap` function a `SwapParams` `_params` parameter is passed along from the `poolManager` which includes a `sqrtPriceLimitX96` value that the user has specified. The user expects that the swap will not pass this limit on execution, however this limit is not honored or referenced at all by the Baseline AMM.

This may cause users to unexpectedly receive much worse executions than they believed possible based on the `sqrtPriceLimitX96` assignment.

— RECOMMENDATION

Consider incorporating the `sqrtPriceLimitX96` in the `BSwap` logic, or perform this check at the end of the `_swap` function in `BSwap`.

Such a check would not be straightforward to implement the same way that `sqrtPriceLimitX96` is implemented in Uniswap though, as the end pool price in `BSwap` is actually the average execution of the trade rather than the infinitesimal last marginal unit execution as it is in Uniswap.

If the average price assignment behavior remains, the meaning of the `sqrtPriceLimitX96` value could be merely adapted.

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/c373edb12b140af393e9d2c7a87bd0df01321466>

MEDIUM **M-12**

ACKNOWLEDGED

Missing Sweep In Borrow Flow

CATEGORY DoS

LOCATION <https://github.com/GuardianOrg/mercury-baselineamm-team2/blob/e3dab29adae84e32a72c8b51634bd9e7cb0173e4/src/components/BCredit.sol#L68-L84>

REVIEW Main Review

DESCRIPTION

The `borrow` function does not call `sweep` to get fetch reserves from the `PoolManager`. However they could be needed to lend them out to the user and in that edge case the function call would DoS.

RECOMMENDATION

Consider calling `sweep` during the `borrow` flow.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-13**

ACKNOWLEDGED

Yield Distributed To Stakers If There Aren't Any

CATEGORY Rewards
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The `distributeFees` function distributes fees to stakers even if there aren't any stakers in the pool yet. However the yield is not lost and instead claimed by future stakers.

— RECOMMENDATION

Consider distributing the remaining yield to the protocol and/or creator if no tokens are staked.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/50fc86bb8ec169b0b08e53008521d6fc3f290ea3/test/fuzzing/FoundryPlayground.sol#L713-L735>

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

poolManager Set Twice

CATEGORY Gas Optimization

LOCATION <https://github.com/GuardianOrg/mercury-baselineamm-team2/blob/e3dab29adae84e32a72c8b51634bd9e7cb0173e4/src/utils/ConfigScript.sol#L24-L25>

REVIEW Main Review

DESCRIPTION

The `poolManager` variable is set twice in the `ConfigScript`. This is redundant.

RECOMMENDATION

Set it only once to save gas.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Fee Claiming May Be Blocked By Zero Amounts

CATEGORY DoS
LOCATION BController.sol
REVIEW Main Review

DESCRIPTION

In some cases the `protocolFee` or `creatorFee` may be assigned to zero and it may also be the case that the reserve token for the pool reverts upon zero value transfers. This prevents the claiming of the other non-zero fee value in the `claimPoolFees` function as the `giveReserves` function does not early return when the provided value to transfer is zero.

RECOMMENDATION

Consider early returning in the `handleOutgoing` as well as `handleIncoming` functions if the `_amount` value is zero.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/cf042c102eff88b2ef70af5fc9bfe1cae5e56017>

LOW **L-03**

ACKNOWLEDGED

Protocol Fee Claim Can Be DoS

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/mercury-baselineamm-team2/blob/e3dab29adae84e32a72c8b51634bd9e7cb0173e4/src/components/BController.sol#L147
REVIEW	Main Review

— DESCRIPTION

The `claimPoolFees` claims both the fees for the creator of the pool and the protocol itself. In case the transfer to the creator fails because the address reverts or is blacklisted the whole transaction reverts. Therefore in that case it is not possible for the Baseline protocol to claim their earned revenue.

— RECOMMENDATION

Consider adding another function to claim protocol fees or using try/catch.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Claimers Miss Yield Harvest

CATEGORY Unexpected Behavior
LOCATION BStaking.sol
REVIEW Main Review

DESCRIPTION

In the claim function in the `BStaking` component there is no invocation to harvest the latest yield from the vault with the `pool.harvest` function. As a result, the latest yield earned by the vault that would go towards the `_pool.pendingYield` is not updated.

As a result, the `targetTps` and thus the `tokensPerSecond_` in the accumulator calculation for the `_sync` call during the claim does not account for this latest accumulated yield. The user is then not credited with the latest accumulator at the time of their claim and therefore is not able to claim the full amount that they should have been.

RECOMMENDATION

Consider invoicing the `pool.harvest` function before calling `_sync` in the `claim` function.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/12595345c4aa8b311b76b501999a26e99f9990a9>

LOW **L-05**

ACKNOWLEDGED

takeReserves Does Not Refund Accidental Eth

CATEGORY Validation
LOCATION VaultLib.sol
REVIEW Main Review

— DESCRIPTION

In the `takeReserves` function, the underlying `handleIncoming` internal function refunds excess ether that may have been accidentally sent in the case when the reserve token is the native token.

However when the reserve token is not the native token but the `msg.value` is still non-zero, this non-zero value is not refunded to the user.

— RECOMMENDATION

Consider refunding any accidental `msg.value` sent in the event where the reserve token is not the native address.

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/cf042c102eff88b2ef70af5fc9bfe1cae5e56017>

LOW **L-06**

ACKNOWLEDGED

Inessential ensureValidCurve Convexity Validation

CATEGORY Validation
LOCATION CurveLib.sol
REVIEW Main Review

— DESCRIPTION

In the `ensureValidCurve` function the `_convexityFactor` uint value is validated to be greater than or equal to 0, however this being a uint value it cannot fail this validation.

— RECOMMENDATION

Consider if the validation should instead validate that the `convexityFactor` is not equal to 0. Otherwise remove the validation as it will never fail by definition.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Misleading Staking Percent

CATEGORY Logical Error
LOCATION BLens.sol
REVIEW Main Review

DESCRIPTION

The `stakingPercent` function reports the staking percentage share of fees as `1 - creatorFeePct`, however the staking percentage is actually represented by: `(1 - protocolFeePct) * (1 - creatorFeePct)`.

RECOMMENDATION

Replace the `stakingPercent` implementation with `(1 - protocolFeePct) * (1 - creatorFeePct)`.

RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Missing Deployment Validation Functions

CATEGORY	Warning
LOCATION	BHooks.sol
REVIEW	Main Review

DESCRIPTION

The [BaseHook](https://github.com/Uniswap/v4-periphery/blob/main/src/utils/BaseHook.sol) contract used by Uniswap gives several functions to manage the deployment and validation of a Hook address such as `getHookPermissions` and `validateHookAddress` which are used in the constructor to validate that the hook has been deployed to the correct address given its permissions.

However the `BHook` contract does not have these functions and lacks the proper deployment address validation in the constructor.

RECOMMENDATION

Consider using the [BaseHook](https://github.com/Uniswap/v4-periphery/blob/main/src/utils/BaseHook.sol) contract from Uniswap to improve the overall security of the `BHook` contract and include these important deployment validations.

RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Fee Claiming May Be Blocked By Protocol

CATEGORY DoS
LOCATION BController.sol
REVIEW Main Review

DESCRIPTION

In the `claimPoolFees` function the `protocolFeeRecipient` address always receives the `protocolFees` amount using `_asNative` as true. Therefore if the `protocolFeeRecipient` address is maliciously or accidentally assigned as an address that cannot accept Ether then the claiming of fees is prevented for all pools where the reserve token is wrapped native.

RECOMMENDATION

Consider sending the fees as the wrapped native token by passing `_asNative` as false when invoking `giveReserves` for the `protocolFeeRecipient`. Otherwise be sure that the `protocolFeeRecipient` is not assigned to an address that cannot accept ether.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/b59f5cc69b0fd2d51909061b8b1e0eb86bc1c0aa>

LOW **L-10**

ACKNOWLEDGED

Frequent Claims Slow Staking Distribution

CATEGORY Unexpected Behavior
LOCATION BStaking.sol
REVIEW Main Review

DESCRIPTION

In the `getAccumulator` function during the `_sync` flow, the tokens per second is affected by the current `pendingYield` of the protocol. The `pendingYield` is decreased over time as stakers interact with the `BStaking` contract and the `_sync` function is invoked which moves the realized `newYield` from the `pendingYield` to the `claimableYield`.

As a result, the time it takes to distribute a certain amount of `pendingYield` is actually affected by the amount of syncs that are done, since the `pendingYield` will be reduced with each sync and the `targetTps` will be dragged down over time, redoing the `tokensPerSecond`.

This may lead to the entire yield amount taking an unexpectedly longer time to fully distribute than anticipated.

RECOMMENDATION

Consider if this behavior is acceptable and if the number of syncs should have an impact on the amount of time it takes to distribute the entirety of pending yield to stakers. If this behavior is not desired, consider refactoring the staking logic such that the rate of distribution is constant.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/dd438016d1195fd9cc288bd4f0421ed42e839a74>

LOW **L-11**

ACKNOWLEDGED

beforeModifyLiquidity Revert Missing

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/mercury-baselineamm-team2/blob/master/src/components/BHook.sol>

REVIEW Main Review

DESCRIPTION

The `BHook` prevents most actions on Uniswap side but does not have a `beforeModifyLiquidity`. Therefore this action is not prevented.

RECOMMENDATION

Consider implementing a `beforeModifyLiquidity` hook which reverts.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Misleading Comment

CATEGORY Logical Error
LOCATION BFactory.sol
REVIEW Main Review

DESCRIPTION

In the `createPool` function the vault is set for the pool with the `setVaultForBToken` at the very end of the initialization. However the comment on the `takeReserves` action mentions that it will `Deposit to vault if set`.

This is slightly misleading as the vault is only set at the end of the function and therefore the vault will not ever be deposited into in this `takeReserves` call.

RECOMMENDATION

Consider removing the `Deposit to vault if set` part of the comment above the `takeReserves` function.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/31b135ce0bba8746fda2c19134cdff779e98be74>

INFORMATIONAL I-02

ACKNOWLEDGED

Redundant Executor Assignment

CATEGORY —
LOCATION BProxy.sol
REVIEW Main Review

DESCRIPTION

In the `executeActions` function if the caller is the admin the `isExecutor` validation is skipped, however in the `constructor` and `acceptAdmin` functions the `executors` mapping is written to for the admin.

This `executors` entry is not necessary, at least for the immediate use-case of the `executeActions` function in the `BProxy` contract, since the `executors` validation is always bypassed for the admin.

RECOMMENDATION

Consider removing the assignment to the `executors` mapping for the admin in the `constructor` and `acceptAdmin` function.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-03**

ACKNOWLEDGED

Unused getCollateralForBorrow Function

CATEGORY Superfluous Code
LOCATION BCredit.sol
REVIEW Main Review

— DESCRIPTION

In the `BCredit` component, the `getCollateralForBorrow` function is unused throughout the codebase.

— RECOMMENDATION

Consider removing the `BCredit` component.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Admin Can Be Added As An Executor

CATEGORY Validation
LOCATION BProxy.sol
REVIEW Main Review

DESCRIPTION

In the `addExecutor` function there is no validation that the admin address cannot be the provided `_executor`. As a result, the admin can accidentally configure themselves as the `_executor` and give themselves temporary `executors` status, overwriting their permanent `executors` status.

RECOMMENDATION

Consider adding validation that prevents the current admin from being the `_executor` address.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Version Limits Upgrades

CATEGORY Warning
LOCATION ActionRouter.sol
REVIEW Main Review

DESCRIPTION

In the `_upgradeComponent` function the `uint8` `newVersion` variable is required to be greater than the previous version. However since the version variables are limited to `uint8` types, a maximum of 255 upgrades can occur for the component.

In some cases this may limit upgrades after a long duration of protocol support.

RECOMMENDATION

Be aware of this limitation and consider using a larger storage type if more than 255 upgrades for a single component should be supported.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

Lacking executeActions Return Value

CATEGORY —
LOCATION BProxy.sol
REVIEW Main Review

DESCRIPTION

In the `executeActions` function the return data from the `delegatecall` operation is not surfaced as a return value from the `executeActions` function.

RECOMMENDATION

Consider if this is expected and if it would be useful to surface the return data if there is any from the `delegatecall` operation as a return value from the `executeActions` function.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

Incorrect Comment

CATEGORY Documentation
LOCATION CurveLib.sol
REVIEW Main Review

DESCRIPTION

In the `getSupplyFromReservesAsk` function the comment at the beginning explains that `on ask curve, targetReserves must be >= currentReserves`, however the target reserves must instead be strictly `>` the `currentReserves` and this is what is implemented by the validation.

RECOMMENDATION

Consider updating the comment at the beginning of the `getSupplyFromReservesAsk` function to read `on ask curve, targetReserves must be > currentReserves`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

ACKNOWLEDGED

Component Contracts Must Report Static Labels

CATEGORY Warning
LOCATION ActionRouter.sol
REVIEW Main Review

— DESCRIPTION

The `ActionRouter` contract depends on component contracts reporting their own labels via the `LABEL` view function.

However if a malicious or malfunctioning component contract reports a `LABEL` value that differs from the value it had reported in the past this leads to an invalidation of the logic in the `ActionRouter`.

— RECOMMENDATION

Be sure to only interact with and support component contracts that return a constant `LABEL` value which cannot change.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09

ACKNOWLEDGED

Admin Can Be Removed As An Executor

CATEGORY Validation
LOCATION BProxy.sol
REVIEW Main Review

DESCRIPTION

The `removeExecutor` function does not prevent the caller from accidentally or purposefully removing the admin as an executor. This action cannot be reversed without re-transferring admin ownership since the `addExecutor` function can only be used to add temporary access, not unlimited access.

RECOMMENDATION

Consider validating that the `_executor` address is not an admin in the `removeExecutor` function.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

Maximum ContractName Label Length

CATEGORY	Unexpected Behavior
LOCATION	Component.sol
REVIEW	Main Review

DESCRIPTION

In the Component contract the `toLabel` function truncates the provided type name to just 32 bytes. Therefore if a contract name exceeds 32 characters then it cannot be fully expressed in the label and may have collisions with other contract names that also exceed 32 characters.

RECOMMENDATION

Be aware of this limitations and keep the component contract names to below 32 character names.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

ACKNOWLEDGED

Lacking Zero Address Checks

CATEGORY Validation
LOCATION BProxy.sol
REVIEW Main Review

DESCRIPTION

In the `constructor` and `transferAdmin` function of the `BProxy` contract there are lacking 0 address checks. For important address values such as the `_newAdmin`, `_admin`, and `_actionRouter`.

RECOMMENDATION

Consider adding zero address validations in the `constructor` and `transferAdmin` function of the `BProxy` contract.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-12**

ACKNOWLEDGED

Unnecessary Return Value

CATEGORY Superfluous Code
LOCATION VaultLib.sol
REVIEW Main Review

DESCRIPTION

The `depositToVault` and `withdrawFromVault` functions state that they will return a `uint256` value in their interface declaration, however only return 0 in the event that the provided `_reserveAmount` is 0.

Furthermore, the return value from this function is not used in the Mercury codebase, therefore it can be removed.

RECOMMENDATION

Consider removing the return value from the `depositToVault` and `withdrawFromVault` functions.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-13

ACKNOWLEDGED

Misleading shareBalance Comment

CATEGORY Documentation
LOCATION StateLib.sol
REVIEW Main Review

DESCRIPTION

In the `Pool` struct it is mentioned that the `shareBalance` is the “Amount of shares if allocated. Equal or greater than `totalReserves`”, however this is not true in several interpretations.

Firstly, if this is to mean that the actual shares amount is greater than the amount of reserves in the pool, this is not true in the case where the share price is higher than 1 reserve and the pool reserves are converted to a number of shares that is less than the amount of reserves deployed.

Secondly, if this is to mean that the amount of shares is always worth more than the `totalReserves` this may also not be the case in the event that the underlying vault is lossy.

RECOMMENDATION

Consider removing the the portion of this comment which states that the amount of shares is “Equal or greater than `totalReserves`” as this may be misleading and does not always hold.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

ACKNOWLEDGED

Risk Of Insolvency And Bank Run

CATEGORY	Warning
LOCATION	—
REVIEW	Main Review

— DESCRIPTION

Baseline rehypothesizes the underlying reserve assets by depositing them into vaults.

The entire logic for deposits, withdrawals, and yield distribution assumes that the `convertToAssets` value is monotonically increasing (as is the case with the sUSDS vault).

If this assumption is broken, it creates an insolvency scenario: the total reserves of the pool could become lower than what is implied by `shareBalance * currentShareValue`.

When lossy vaults are considered, these assumptions no longer hold.

— RECOMMENDATION

Be aware of this risk: underlying vaults integrated with Baseline should always be monotonically increasing in value, similar to the sUSDS vault.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-15**

ACKNOWLEDGED

Rounding Risk

CATEGORY	Warning
LOCATION	VaultLib.sol
REVIEW	Main Review

DESCRIPTION

Baseline tracks `shareBalance` for rehypothesized vault assets. On deposits, the balance is incremented, and on withdrawals, it is decremented:

```
_pool.shareBalance += newShares;  
_pool.shareBalance -= newShares;
```

However, vaults round down on deposit and up on withdrawal. Because of this, the two operations may not perfectly cancel out due to precision loss.

This discrepancy would only materialize and cause revert for last user if the vault were fully emptied, which is an unlikely scenario.

RECOMMENDATION

Beware of this sceanario.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-16**

ACKNOWLEDGED

Accumulator Update Skips

CATEGORY	Warning
LOCATION	BStaking.sol (Global)
REVIEW	Main Review

DESCRIPTION

The accumulator is only updated once per block. If no time has elapsed, the function returns early:

```
uint256 timeElapsed = block.timestamp - staking.lastUpdated;  
if (timeElapsed == 0) return (accumulator_, 0, tokensPerSecond_);
```

This works as intended for vaults like sUSDS, where yield accrual is block-time based with a constant rate.

However, in dynamic vaults (e.g., rebalancing vaults or lending vaults accruing liquidation penalties), yield can accrue at arbitrary times within a block. Since Baseline skips accumulator updates when multiple actions occur in the same block, an attacker can exploit this by:

1. Calling `claim` for any user before yield accrues in the vault (forcing a sync).
2. Staking afterwards, still within the same block.
3. Waiting for subsequent blocks to unstake.

This effectively sandwiches the yield generated by the vault during that block, misattributing it to the attacker.

Baseline's `timeToAdapt` mechanism significantly reduces this risk, since it forces gradual adaptation:

```
uint256 adjustment = FixedPointMathLib.min(  
(delta * timeElapsed) / meta.timeToAdapt,  
delta  
);
```

Still, in cases where yield between periods $a \rightarrow b$ and $b \rightarrow c$ differs, attackers can profit if yield from $a \rightarrow b$ is higher, since they only bear yield distribution for $b \rightarrow c$.

I-16 Accumulator Update Skips

CONTINUED

— RECOMMENDATION

Beware of this, when rehypothecating into vaults with dyanmic yield accrual.

Thing is there is no solution to this problem besides what baseline already doing, even if sync is enforced on each action, attacker can just sandwich with stake - vault action - unstake.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-17**

ACKNOWLEDGED

Redundant Check

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/mercury-baselineamm-team2/blob/e3dab29adae84e32a72c8b51634bd9e7cb0173e4/src/libraries/CurveLib.sol#L270
REVIEW	Main Review

DESCRIPTION

The `ensureValidCurve` checks that a `uint256` is not < 0 . This can never happen anyway therefore this check is redundant.

RECOMMENDATION

Adjust or remove the redundant check.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-18**

ACKNOWLEDGED

Vault Loss Will Break Calcs

CATEGORY Warning
LOCATION Global
REVIEW Main Review

— DESCRIPTION

Major issues could arise in case the protocol decides to use a vault with a index which is not strictly increasing (for example as bad debt is socialized among vault participants).

The main reason for that is the `harvest` flow which will realize and safe yield from the vault as fixed profit. However the same is not done for loss in the vault. Therefore if profit is already tracked by Baseline but loss in the vault occurs afterwards the system will continue to calculate with tokens it doesn't own.

— RECOMMENDATION

Be aware to not use such vaults.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-19**

ACKNOWLEDGED

Lending Is Capital Inefficient

CATEGORY	Rewards
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The Baseline Credit system lends out reserves for a fixed fee, the origination fee which currently is set to 0.1%.

The system also allows to supply all the reserves to Aave or a similar protocol. In this case lending out reserves is probably capital inefficient for the protocol as the yield from Aave will outperform the 0.1% flat fee in a short period of time.

— RECOMMENDATION

Consider to rethink this part of the system or increase the lending fee.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-20**

ACKNOWLEDGED

Variable Missing In NatSpec

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/mercury-baselineamm-team2/blob/e3dab29adae84e32a72c8b51634bd9e7cb0173e4/src/components/BSwap.sol#L458-L469>

REVIEW Main Review

DESCRIPTION

The `_price` variable is missing in the `recordSwap` NatSpec.

RECOMMENDATION

Consider making sure that all NatSpecs are complete.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-21**

ACKNOWLEDGED

Unused Code

CATEGORY	Superfluous Code
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

There are unused variables in multiple parts of the code like for example:

- The `meta` variable in the `harvestYield` function
- The `staking` variable in the `leverage` function
- The `pool` variable in the `_repay` and `updateBlvPrice` function

— RECOMMENDATION

Consider remove unused code.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.